

Lab

Object-oriented programming

Lab: OOP — getting started

Objective

The primary objectives for this lab are:

- To create and use references.
- To consolidate on passing by value and passing by reference.

Part 1 — Creating and using reference types

1. Back in the **labs** project, add a new class called **Program** with **main()** method in a package called **lab2**.
2. Create another class called **Account** in the **lab2** package with following private fields:
`int id;`
`String owner;`
`double balance;`
and methods:
`void Deposit(double amount) { }`
`void Withdraw(double amount) { }`
3. Make sure no one can deposit a negative amount of money! Display an appropriate error message.
4. Make sure no one can withdraw money they don't have.
Display an appropriate error message.
5. Create a **constructor** for the Account class to set its **ID**, **owner**, and starting **balance**.
6. Create a method called **getDetails()** to return details of the account as a formatted String.
7. In the **main()** method, create an instance of the Account class.
8. Invoke its **deposit()** and **withdraw()** methods.
9. Call the Account's **getDetails()** method and print the result to make sure your code has had the expected result.

Part 2 — Creating an array of reference types

1. Back in the **main()** method, create an **array of 3 Account** references.
2. Write an Enhanced for loop to iterate through the three accounts and display their details by calling their **getDetails()** method.

Part 3 — Passing reference types to a method

In this section, you will explore the concept of passing reference types between methods. This is a crucial topic to understand, as it has significant implications for both program behaviour and security.

Step-by-step

1. In the Account class, create a new method as:
public void **AddInterest()**
Write code in this method to add 2.5% interest to the balance.
2. Add code in **main()** to create an account called **myAccount**, with a balance of £100.
3. Call the **addInterest** method of the **myAccount** instance.
4. After the call, display the details of **myAccount** by using its **getDetails()** method.
Did the balance change?
5. Back in the **main()** method, create another Account reference called **partnerAccount** and set it to the **myAccount** instance as:
Account **partnerAccount** = **myAccount**;
6. Call the **partnerAccount's** **addInterest()** method.
7. Call **myAccount's** **getDetails()** method.
Did the balance change?

Let's do another experiment

8. Create a method in the **Program** class (just under the **main()**)
static void **processAccount**(**Account** **acc**){
 acc.addInterest();
}
9. Back in **main()**, pass the **myAccount** instance that you created earlier to the **processAccount** method as parameter.

10. Call **myAccount's getDetails()** method. Did the balance change?

11. Let's do another important experiment.

12. Create another method under the main() method as:

```
private static void incInt(int x) {  
    x++;  
}
```

13. Add code to the main() method to create an integer like

```
int k=100;
```

14. Call the **incInt** method and pass it **k**.

15. After the call, print the value of **k**.

Did **k** change after the call?

Explanation

Primitive data types (built into the Java language) are always passed by value, which means a copy of the value is passed to the method. As a result, modifying a variable (like **x**) does not affect the original variable (**k**). Renaming the method parameter to **x** does not change the outcome!

For reference types (like **myAccount**), the situation is slightly different. While they are also passed by value, the value being passed is the memory address of the object. #

This means that when you call a method like **processAccount(Account acc)** and pass a reference to an account, you are actually passing the address of **myAccount** in memory. Inside the method, the **acc** reference will point to **myAccount**.

Consequently, any changes made via the **acc** reference will directly impact **myAccount**.

Optional exercise (if you have time)

- Create a class called **Bank**.
- Add an Array in Bank for Storage:
 - Inside the Bank class, declare an array of type Account with a fixed storage capacity of three to store just three Account objects for our small bank.
- Create a method called **addAccount**: with the following signature:
 - **Account** addAccount(**String** owner, **double** balance)
 - The method should:

- Create a new Account object with a unique ID (IDs should be sequential: 1, 2, 3).
 - Add the Account to the array if there is space.
 - Return the Account object if successful.
 - Return **null** if the array is full (more than three accounts).
-
- Create a method called **getAccount** Method with the following signature:
 - **Account** getAccount(**String** owner)
 - The method should:
 - Search the array for an account with the specified owner name.
 - Return the corresponding **Account** object if found.
 - Return **null** if no account with the given owner exists.
-
- Test your Bank class to:
 - Create three accounts.
 - Look for an account with the name of a valid customer.
 - **Check null is return:**
 - if more than three Accounts are added.
 - when searching for an unknown customer name.



QA.com